

PU-Based Quality Classifier for LLM Training Texts

Kacper Paczutkowski ¹  and Konrad Furmańczyk ¹ 

¹ Institute of Information Technology, Warsaw University of Life Sciences, Warsaw, Poland;

kacper_paczutkowski@sggw.edu.pl; konrad_furmanczyk@sggw.edu.pl

* Correspondence: kacper_paczutkowski@sggw.edu.pl

Abstract

We study positive-unlabeled (PU) learning for classifying Polish texts intended for large-language-model training. The paper presents a Python reimplement of the training and evaluation pipeline from earlier work, together with preprocessing tailored to the SpeakLeash corpora used in our experiments. We evaluate six PU methods: naive, clust, strict-lassclust, non-strict-lassclust, lassojoint, and spy under both classic and MVC non-SCAR labelling schemes. Across the benchmark, the best F1 reaches 0.906 on open subtitles corpus under classic labelling and 0.900 under MVC labelling; the largest absolute gain over the Naive baseline is 0.811 F1 points on ulotki medyczne and 0.758 on plwiki in the classic setting. The experimental workflow supports multiple labelling rates calculation and repeated random seeds, and evaluates models with AUC, PR-AUC, and F1 as the main metrics. Our study focuses on six SpeakLeash datasets spanning different domains: plwiki, open subtitles corpus, job offers corpus, ISAP corpus, ulotki medyczne, and wolne lektury corpus. For each dataset, we construct a binary PU setup from the quality field, where HIGH is treated as positive and LOW is negative, while other values are removed. The resulting preprocessed tables retain only the selected linguistic and structural features, rescale numeric variables to the $[0, 1]$ range, and remove the raw text column.

Keywords: positive-unlabeled learning; text quality; Polish texts; SpeakLeash; natural language processing; data cleaning

1. Introduction

Positive-unlabeled learning is a natural fit for text quality assessment when only a subset of high-quality examples is explicitly identified and the remaining documents are partially observed. This scenario arises naturally whenever collecting negative labels is costly or semantically ambiguous – a situation that directly characterizes large-scale text corpora such as those studied here [4].

The theoretical foundations of PU learning have been well-established. Bekker and Davis [4] provide a comprehensive survey covering label-frequency estimation, class-prior estimation, and algorithm design under partial supervision. Liu et al. [5] formalized this structure for text classification and introduced the Spy technique, in which a fraction of labeled positives is temporarily moved into the unlabeled set to calibrate a threshold for reliable negative extraction. Nigam et al. [7] showed that even a small labeled set combined with unlabeled documents via Expectation-Maximization substantially reduces classification error – an early demonstration that the unlabeled majority carries exploitable signal. The PU framework has since been applied across diverse domains: Fusilier et al. [6] showed that a PU classifier trained solely on confirmed fake hotel reviews achieves

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Appl. Sci.* for possible

open access publication under the

terms and conditions of the [Creative](#)

[Commons Attribution \(CC BY\)](#) license.

F-measure on par with fully supervised baselines, and Furmańczyk et al. [2,3] developed propensity-based and cluster-assisted logistic methods that form the direct methodological basis of the present work. Recent work has extended PU learning to settings with extremely few labeled positive examples. Dai et al. [19] derived theoretical conditions under which reliable learning remains possible and proposed the ScalePU framework for such highly label-scarce scenarios. A complementary line of research concerns class-prior estimation under distribution shift. Mielniczuk et al. [24] proposed a kernel-embedding estimator for PU data that explicitly accounts for prior shift and established non-asymptotic theoretical guarantees for the estimator. More recently, Liu et al. [20], an in-context learning framework based on pretrained transformers that performs PU classification without task-specific fine-tuning. These recent developments demonstrate the rapid expansion of PU learning toward both stronger theoretical guarantees and modern foundation-model architectures. However, they have not yet been systematically evaluated for stylometric quality assessment of Polish LLM training corpora. Mansouri et al. [21] provide the first theoretical analysis of the label complexity of active PU learning, studying how an adaptive querying strategy can reduce the number of labels needed to recover a reliable classifier.

A critical practical consideration is that the labeling mechanism in quality-annotation pipelines is unlikely to satisfy the *Selected Completely At Random* (SCAR) assumption. Formally, SCAR requires $e(\mathbf{x}) := P(S = 1 \mid Y = 1, \mathbf{x}) = c$ for some constant c independent of $\mathbf{x}$ – that is, every positive instance is equally likely to be labeled regardless of its features. In practice, however, annotators preferentially label visually prominent documents – those with higher lexical density or greater length – making the labeling probability feature-dependent and thus violating the SCAR assumption [4]. This non-SCAR regime motivates the use of propensity-based labeling schemes that explicitly model $e(\mathbf{x})$ [3]. From this perspective, document-quality annotation for LLM training texts can be viewed as a PU learning problem under a non-SCAR labeling mechanism: only a subset of truly high-quality documents receives an explicit positive label, while the remaining pool is left unlabeled and therefore contains a mixture of high- and low-quality texts. The probability that a high-quality document is annotated as such then depends on its observable attributes $\mathbf{x}$ (for example length, lexical density, or domain), which matches the instance-dependent propensity setting $e(\mathbf{x}) = P(S = 1 \mid Y = 1, \mathbf{x})$ emphasized in recent PU learning work [4].

The practical stakes of quality filtering in Polish NLP are illustrated by the SpeakLeash-based Bielik model family [10,11,22]: successive versions (7B v0.1, 11B v2, 11B v3) were trained on best quality SpeakLeash corpora and demonstrated that curated Polish data is essential for competitive downstream performance. Penedo et al. [8] reached a structurally identical conclusion for English web text: their FineWeb-Edu classifier – trained on a small LLM-annotated positive seed and a large unlabeled CommonCrawl pool – is, in effect, a PU learner, even though it is not explicitly framed as such. More recently, Emami Gohari et al. [15] demonstrated at scale that combining heuristic and classifier-based quality signals into a unified pipeline (GneissWeb) substantially improves LLM pretraining outcomes, reinforcing the importance of principled data-quality filtering for both English and multilingual corpora. Negoifa et al. [17] show that diversity-aware quality filtering strongly benefits Romanian LLM pretraining, highlighting the relevance of classifier-based filters for under-resourced languages beyond Polish. Li et al. [9] caution, however, that quality filters anchored to a fixed high-quality reference set introduce selection bias and reduce corpus diversity, because the filtering criterion is implicitly shaped by the characteristics of the seed collection rather than by the downstream task. Nait Saada et al. [18] further argue that classifier-based quality scores can conflate changes in corpus composition with genuine quality improvements, challenging the view that classifier-based quality filtering (CQF) always captures an intrinsic notion of data quality. Langlais et al. [23] address the data

availability problem from a different angle, releasing Common Corpus – the largest open and fully licensed collection for multilingual LLM pretraining – which covers Polish among dozens of European languages and establishes a reproducible baseline for open-science LLM research.

This observation reinforces the need for principled PU methods that explicitly model the labeling propensity $e(\mathbf{x})$ and decouple it from the classification objective – exactly the distinction drawn by the non-SCAR framework evaluated in the present work. In this paper we study whether dedicated PU methods can classify Polish texts in a reproducible way across heterogeneous domains and data sizes. The full experimental pipeline is implemented in Python¹ and separates labelling, preprocessing, and model training into distinct stages.

1.1. Related Work and Contributions

Modern data filtering for LLM training increasingly uses classifier-guided pipelines such as FineWeb-Edu [8], the GneissWeb framework [15], and scaling-aware quality filters such as ScalingFilter [9]. Those approaches motivate our study, but they still do not answer the PU-specific question of how to model positive-only labeling under non-SCAR conditions for Polish corpora. Recent work on low-resource languages demonstrates that similar classifier-based quality filtering pipelines can substantially improve pretraining data for Romanian [17]. At the same time, Nait Saada et al. [18] provide a critical analysis of CQF-style filters for LLM pretraining, showing that classifier scores may not always correspond to intrinsic data quality; we acknowledge this as a limitation of classifier-based filters and treat our PU classifiers primarily as tools for prioritizing data rather than as complete quality metrics. Langlais et al. [23] complement this picture by providing a large-scale multilingual resource with explicit provenance metadata, enabling future work to benchmark quality classifiers under reproducible and ethically vetted conditions. On the PU learning side, Wang et al. [16] recently proposed a standardized evaluation framework for PU algorithms that highlights the importance of realistic experimental conditions and reproducible benchmarking – a concern shared by the present work. Dai et al. [19] complement this perspective with a theoretical analysis of PU learning under extreme label scarcity and a variance-regularized ScalePU estimator. Mielniczuk et al. [24] address the related problem of class-prior estimation when the target distribution shifts, which is directly relevant to our cross-domain evaluation setting. The PUICL model of Liu et al. [20] demonstrates that in-context learning can be applied to PU classification without dataset-specific training, a direction that could further reduce the annotation overhead in future extensions of this work. Mansouri et al. [21] provide formal guarantees on the label complexity achievable by an adaptive querying strategy in the PU setting, motivating future work on active annotation for SpeakLeash corpora.

The concrete contributions of this paper are:

- First comprehensive benchmark of PU-learning methods for Polish LLM training corpora.
- Evaluation under two different non-SCAR labeling mechanisms.
- Cross-domain study across six heterogeneous SpeakLeash datasets.
- Practical recommendations for PU-learning deployment in LLM data filtering.
- Open-source Python implementation enabling reproducible research.

Table 1 summarizes the substantive differences between the previous our non-SCAR study and the present manuscript.

¹ Selected classification scripts were rewritten by the authors from their earlier R implementations (<https://github.com/kapacc/mdai25-clusters-scripts>) into Python with assistance from GitHub Copilot, mainly using a GPT-Codex model. The current codebase is available at <https://github.com/kapacc/pullm>.

Table 1. Substantive novelty of the present manuscript relative to the our earlier non-SCAR study.

Aspect	Earlier work	Present manuscript
Scope	One PU benchmarking setup centered on clustering and logistic modeling under non-SCAR assumptions	Six PU methods compared under two non-SCAR label-generation schemes across six SpeakLeash corpora
Data handling	Earlier pipeline and narrower experimental scope	Reimplemented in Python with shared preprocessing, repeated seeds, and unified evaluation protocol
Method coverage	Cluster-assisted and logistic non-SCAR variants	Adds Naive, Strict-LassClust, Non-Strict-LassClust, LassoJoint, and Spy as direct comparators
Interpretation	Method comparison focused on algorithmic behavior	Adds dataset-level discussion, limitations, and practical method-selection guidance for LLM filtering
Reporting	Earlier benchmark summary	Expanded tables, stability analysis, runtime summaries, and reviewer-oriented interpretation

Table 2. Comparison of selected related work and the present study.

Work	Application Domain	PU Learning	Non-SCAR	LLM Data Filtering
Liu et al.	Text classification	✓	×	×
FineWeb	Web filtering	Implicit	×	✓
Ociepa et al.	Polish LLM	×	×	✓
This work	Polish LLM	✓	✓	✓

Throughout this work, we adopt the *single-scenario* formulation of PU learning [4]. Specifically, we assume that the latent triples (X_i, Y_i, S_i) , $i = 1, \dots, n$, are i.i.d. realizations of the random vector (X, Y, S) . However, only the pairs (X_i, S_i) are observed, where $S_i = 1$ implies $Y_i = 1$, while observations with $S_i = 0$ are unlabeled and originate from a mixture of positive and negative instances. This stands in contrast to the *case-control* scenario, where separate i.i.d. samples from $p(\mathbf{x} \mid Y = 1)$ and from the marginal $p(\mathbf{x})$ are available. The single-scenario setting is the natural one for our use case: the SpeakLeash corpora arrive as a single collection in which only a fraction of high-quality documents carries an explicit HIGH label, while the remaining documents are left unlabelled regardless of their true quality.

2. Objectives and Methods

The primary objective of this work is to investigate whether positive–unlabeled (PU) learning methods can be effectively applied to text quality classification of Polish LLM training corpora. To achieve this objective, we evaluate six representative PU learning methods using the SpeakLeash corpora. We implemented a comprehensive benchmarking pipeline that controls for both the labelling mechanism and the estimation of the class-prior proxy c , enabling direct comparison of PU methods under non-SCAR settings.

Table 3. Characteristics of the PU learning methods considered in the benchmark.

Method	Type	Handles non-SCAR	Computational Cost
Naive	Baseline PU	No	Low
Clust	Cluster-based PU	Yes	Medium
LassoJoint	Propensity-aware PU	Partial	Medium
Spy	Reliable-negative PU	Partial	Medium

2.1. PU Learning Methods

We evaluate six PU methods with increasing algorithmic complexity:

1. **Naive**: Standard logistic regression trained directly on the observed label S as a binary target, treating labeled positives ($S = 1$) as the positive class and all unlabeled instances ($S = 0$) as negatives, without any correction for the fact that the unlabeled pool contains a mixture of true positives and true negatives. As a result, the model implicitly assumes $e(\mathbf{x}) = 1$, i.e. that every positive instance is labeled – a strong and generally violated assumption that causes the predicted scores to be systematically downward-biased for true positives hidden in the unlabeled set.
2. **Clust** [2]: Applies an iterative “pecking-order” procedure to estimate logistic regression coefficients; the number of iterations and the pecked fraction are treated as parameters, set to 5 iterations and full labeled-positive set per iteration in this experiment. In each iteration, a random subset of labeled positive instances ($S = 1$) is temporarily moved into the unlabeled pool (in our work param of `pecked_part = 1`); the combined set is standardized (Z-score) and partitioned into two clusters via K-means ($K = 2$). The cluster with the higher proportion of labeled positives is treated as positive, and the remaining cluster as negative. A logistic regression is then fitted on the resulting pseudo-labeled data, and final coefficients are obtained by averaging estimates across all iterations.
3. **Strict-LassClust** [2]: Combines Clust (details above) with L1 regularization (Lasso). Coefficients are averaged across clusters using strict averaging (only features selected in all clusters are retained).
4. **Non-Strict-LassClust** [2]: Similar to Strict-LassClust but uses non-strict averaging (averaging across all clusters regardless of selection consistency).
5. **LassoJoint** [13]: Combines L1-regularized feature selection with a joint PU logistic model fitted on the selected support. In the first step, a Lasso logistic regression with cross-validated penalty (λ selected via 10-fold CV) is used to identify informative features; only those with $|\hat{\beta}_j| > \delta$ (where $\delta = 0.5\lambda$) are retained. In the second step, a joint logistic model $R(\mathbf{b}, c)$ is fitted on the selected features by minimizing the negative log-likelihood

$$-\sum_i [S_i \log(c \sigma(\mathbf{x}_i^\top \mathbf{b})) + (1 - S_i) \log(1 - c \sigma(\mathbf{x}_i^\top \mathbf{b}))],$$

which simultaneously estimates the coefficient vector $\mathbf{b}$ and the class-prior proxy $c = \mathbb{E}[S \mid Y = 1]$.

6. **Spy** [5]: Uses the Spy technique to extract reliable negatives from the unlabeled set, then trains a classifier on the augmented labeled pool. A fraction of labeled positive examples is temporarily moved into the unlabeled set, and predictions on these *spy* samples calibrate a classification threshold for reliable negative extraction.

Deep generative PU architectures, including GAN-based models [14] and variational-autoencoder approaches, are most often analyzed in a case-control sampling scheme, where one observes separate samples from the positive class and from the mixture distribution over the general population. This assumption is less realistic for our SpeakLeash use case, which follows the single-scenario design with a single partially labeled sample. In principle, the single-scenario regime with selection bias can be addressed by variational methods such as the VAE-PU+OCC model of Wawrzęńczyk and Mielniczuk [12], but deploying such architectures on our largest corpora would be prohibitively expensive computationally, especially in terms of GPU memory and wall-clock training time.

Table 4. End-to-end benchmark workflow used in the manuscript.

Stage	Input	Output / Purpose
Data selection	Raw SpeakLeash corpora	Six datasets spanning law, dialogue, job offers, encyclopedia, medical leaflets, and prose
Preprocessing	Quality metadata and 22 stylometric features	Binary HIGH/LOW PU target, min-max scaled features, raw text removed
Non-SCAR labelling	Preprocessed training pool	Classic or MVC labels with target c_{calc} and seed control
Model fitting	Labeled training split	Six PU classifiers fitted on the same feature space
Evaluation	Held-out test split	AUC, PR-AUC, and F1 computed per dataset and strategy
Interpretation	Aggregated results	Dataset-specific discussion, runtime analysis, and practical method guidance

The methods studied here were chosen to cover the most relevant trade-offs for this setting. Naive provides the strongest possible lower bound. Clust and the two LassClust variants test whether clustering and sparse logistic fitting can recover structure from weakly labelled features. LassoJoint is the most direct propensity-aware method in the set. Spy represents a classical PU baseline that is still popular in text classification. Together, the six methods span simple, cluster-assisted, sparse, and propensity-aware families without introducing a separate neural architecture family that would require a very different computational regime.

2.2. Benchmark Workflow

Table 4 gives a compact summary of the end-to-end benchmark workflow, from raw SpeakLeash data to final evaluation.

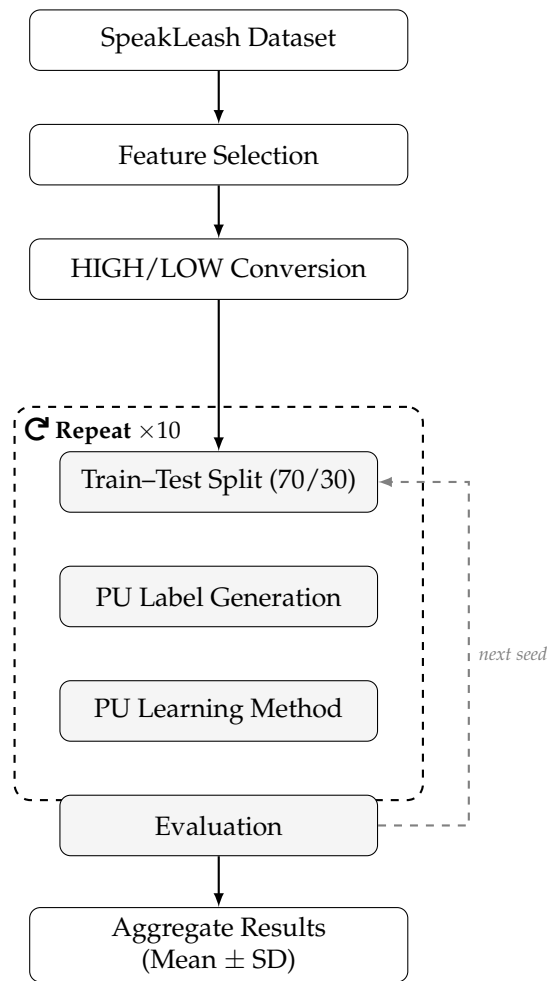


Figure 1. Benchmark pipeline for a single dataset and labelling strategy.

2.3. Experimental Design and Evaluation

Each method is evaluated across multiple random seeds (10 repetitions) and three target labeling rates ($c_{calc} \in \{0.2, 0.5, 0.8\}$). We use the union-train protocol. For each seed and each dataset, we first draw up to 10,000 examples from the preprocessed table (if a dataset is smaller, all rows are kept), preserving the class composition induced by Y . Next, each sampled dataset is split into train and test parts using a stratified 70/30 split. The training parts from all datasets are then concatenated into one shared PU training set, on which non-SCAR labeling is generated and all PU methods are fitted once. Evaluation is then performed separately for each dataset: predictions are sliced back to the corresponding test subset, and AUC, PR-AUC, and F1 are computed per dataset. The experimental pipeline was implemented using dedicated Python scripts. The current codes used in this study are publicly available in the paper repository.²

The sample size of 10,000 examples per corpus was chosen to balance computational tractability with statistical representativeness. While modern LLM data filters operate on corpora orders of magnitude larger, the present benchmark is designed to evaluate the methodological behaviour of PU classifiers under controlled conditions – with fixed feature spaces, shared preprocessing, and repeated seeds – rather than to simulate production-scale filtering. The 10,000-example cap ensures that results remain comparable across corpora of very different sizes (from ~6 000 to ~1.4M rows) while keeping wall-clock training time

² <https://github.com/kapacc/pullm>

Table 5. Comparison of labelling strategies used in the benchmark.

Property	Classic (non-SCAR)	MVC (non-SCAR)
Type	non-SCAR	non-SCAR
Mechanism	Logistic propensity $e(x) = \sigma(\alpha + \theta^\top x)$; $S \sim \text{Bernoulli}(e(x))$ for $Y = 1$ only	Rank-based scoring using sum of top-variance features; $S = 1$ for $Y = 1$ below rank threshold
Dependence of S on X	Yes – linear in features (via $\theta^\top x$)	Yes – nonlinear, rank-based
Calibration of c_{calc}	Grid search over $\alpha \in [-10, 10]$ minimising $ \mathbb{E}[e(x) Y = 1] - c $	Linear interpolation over quantile levels $\{0.1, 0.3, 0.5, 0.7, 0.9\}$
Used in benchmark	<code>labelling_strategy=classic</code>	<code>labelling_strategy=mvc</code>

manageable across six methods, three labelling rates, and ten seeds. We discuss this choice as a limitation in Section 7.

2.4. Background: The Bielik Quality Classification Use Case

The original motivation for PU learning in Polish NLP stems from the Bielik model family. The Bielik 7B v0.1 model was adapted from Mistral 7B v0.1 due to limited access to high-quality Polish data. During pre-training preparation, 9,000 documents were manually annotated with quality labels (HIGH, MEDIUM, LOW), and 266 stylometric features were computed per document (verb/noun frequencies, sentence counts, punctuation patterns). An XGBoost classifier was then used to classify the full corpus, retaining only documents predicted to be high quality for final LLM training. This historical approach motivates the PU framework: only a seed set of confirmed high-quality documents is available, while the remaining pool remains partially observed, making dedicated PU methods essential for scalable and principled quality assessment. The subsequent Bielik v3 series [22] extended this approach by replacing the universal Mistral-based tokenizer with a Polish-optimized vocabulary, further illustrating how data-quality decisions interact with architectural choices in the development of language-specific LLMs.

Both labelling strategies respect the class structure ($Y \in \{0, 1\}$) but differ fundamentally in how they model the selection propensity $e(\mathbf{x}) = P(S = 1 | Y = 1, \mathbf{x})$ and calibrate the achieved class-prior proxy $c = \mathbb{E}[S | Y = 1]$ to match a target value c_{calc} .

The **Classic (logistic) strategy** assumes a linear propensity model $e(\mathbf{x}) = \sigma(\alpha + \theta^\top \mathbf{x})$, where the propensity score uses all available variables and α is calibrated via grid search over $[-10, 10]$ to minimize $|c_{\text{calc}} - c|$. This approach is stochastic: each run generates labels independently via Bernoulli sampling, leading to non-zero variance across random seeds. It follows established PU simulation practice [3]. In the Classic strategy we use a homogeneous coefficient vector $\theta = (1, \dots, 1)^\top \in \mathbb{R}^p$ (i.e. $\theta_j = 1$ for all j), so that the linear predictor reduces to $\alpha + \sum_{j=1}^p x_j$. The intercept α is selected on a uniform grid of 401 points spanning $[-10, 10]$ to minimise $|\mathbb{E}[e(\mathbf{X}) | Y = 1] - c_{\text{calc}}|$, where the expectation is approximated by the sample mean over positive instances in the training set. Once the optimal $\hat{\alpha}$ is found, selection labels are drawn as $S_i \sim \text{Bernoulli}(e(\mathbf{x}_i))$ independently for each $Y_i = 1$, while $S_i = 0$ is set deterministically for all $Y_i = 0$.

The **MVC (Most Variable Columns) strategy** is a deterministic, rank-based mechanism based on feature variance ranking. In this work samples are scored by summing only the two features with the highest variance and then ranked. A linear threshold is computed by interpolation at quantile levels $\{0.1, 0.3, 0.5, 0.7, 0.9\}$ to match the target c_{calc} . This method was introduced in [2].

3. Datasets

SpeakLeash is an open-source ecosystem centered on Polish-language AI resources. According to the official package documentation, it provides a lightweight Python library for downloading and inspecting Polish datasets together with dataset manifests and meta-data access [1].

Our experiments use six SpeakLeash datasets selected to cover distinct domains and scales [1]. The selected corpora span encyclopedia-like content, dialogue, job offers, legal documents, medical texts, and literary prose. This mix allows us to test PU methods on both shorter and longer documents, as well as on sources with different vocabulary richness and style.

- `plwiki` is a Polish Wikipedia mirror containing encyclopedia entries across diverse topics; the dataset combines both longer structured articles and shorter stub entries.
- `open_subtitles_corpus` consists of subtitles extracted from film and television dialogues, yielding shorter utterances with informal and conversational language patterns.
- `job_offers_pl_corpus` aggregates job listings from online employment portals, typically featuring semi-structured content with repetitive formatting and domain-specific terminology.
- `ISAP_corpus` comprises legal and administrative documents from Polish public institutions, characterized by long, formally structured sentences with specialized vocabulary.
- `ulotki_medyczne` contains medical information leaflets and health-related texts, featuring technical terminology and precise descriptive language.
- `wolne_lektury_corpus` is a collection of Polish literary prose from the public domain, including novels and short stories with diverse narrative styles and rich vocabulary.

For each dataset, we use the quality metadata provided with the corpus to construct the PU target. Documents labeled `HIGH` are mapped to the positive class ($Y = 1$), documents labeled `LOW` are mapped to the negative class ($Y = 0$), and all remaining quality values are discarded. We then keep only the selected feature columns used by the benchmark pipeline, remove the raw `text` column, and rescale all numeric variables to the $[0, 1]$ range using min-max scaling. A detailed description of all 22 linguistic and stylometric features is provided in Table 13 of the Supplementary Material. Importantly, in line with the SpeakLeash curation perspective, these quality labels primarily reflect usefulness for LLM training (e.g., clarity, consistency, and preprocessing suitability), rather than artistic merit or stylistic richness.

The exclusion of `MEDIUM` is intentional rather than accidental. Those documents are ambiguous by construction and do not provide a clean negative class, so keeping them inside a binary `HIGH/LOW` benchmark would entangle class-label noise with PU noise. We therefore use the binary split as a controlled experimental proxy for the practical filtering task, but we interpret the results as methods for prioritizing high-quality data rather than as a complete quality-ranking solution. This is especially important for corpora such as `ISAP_corpus`, where the positive class is extremely rare and any method can become sensitive to small changes in the label mechanism.

Table 6 summarizes the datasets used in the current scope and combines corpus metadata with PU-related summary statistics. Mean Abs Corr denotes the average absolute Pearson correlation between each feature.

Figure 2 presents a condensed comparison of shared feature distributions for all six pre-processed SpeakLeash corpora used in this benchmark (`plwiki`, `open_subtitles_corpus`, `job_offers_pl_corpus`, `ISAP_corpus`, `ulotki_medyczne`, `wolne_lektury_corpus`). The left panel shows per-feature medians, while the right panel shows interquartile ranges

Table 6. SpeakLeash datasets used in the latest benchmark run, with corpus metadata and PU summary statistics.

Dataset	Text type	Raw rows	Preprocessed rows	Neg Inst	Pos Inst	Pos %	Mean Abs Corr
ISAP_corpus	Legal	83 846	83 708	83 008	700	0.84	0.30
job_offers_pl_corpus	Job offers	895 651	868 899	722 770	146 129	16.82	0.32
open_subtitles_corpus	Dialogue	63 153	49 232	17 677	31 555	64.09	0.31
plwiki	Encyclopedia	1 469 920	1 422 806	829 365	593 441	41.71	0.34
ulotki_medyczne	Medical	19 336	9 787	9 180	607	6.20	0.50
wolne_lektury_corpus	Literary prose	6 619	6 087	3 786	2 301	37.80	0.39

(IQR). Two patterns are visible: first, central tendencies differ most strongly for ratio-style features such as lexical density, noun ratio, and verb ratio; second, several count-like features have very low IQR after min-max scaling, indicating narrow within-dataset spread despite clear between-dataset differences.

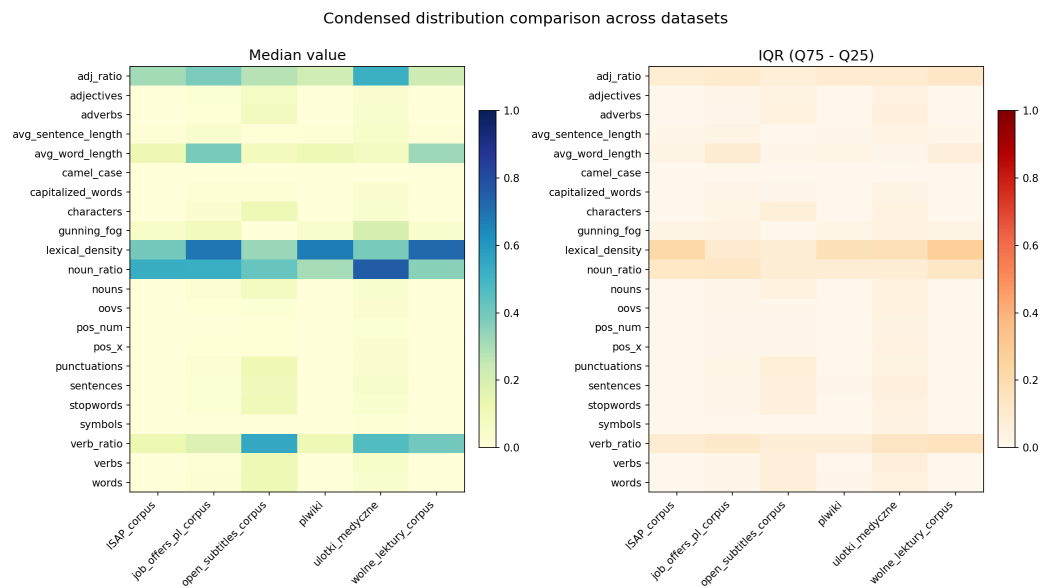


Figure 2. Condensed comparison of shared feature distributions for all six preprocessed SpeakLeash corpora used in this benchmark. Left: feature medians. Right: IQR ($Q_{0.75} - Q_{0.25}$).

In conclusion, Fig. 2 confirms that the visualized corpora are not interchangeable in feature space, even after uniform preprocessing and min-max scaling. The largest cross-dataset contrasts are concentrated in ratio-based variables, which suggests that stylistic and lexical composition differs more strongly than raw count intensity. At the same time, low IQR values for many count-like variables indicate internally stable feature profiles within each corpus, supporting dataset-specific benchmarking rather than pooling all sources into one training distribution.

The same interpretation applies to the binary PU setup. The dataset statistics show that ISAP_corpus contains only 0.84% positive instances, so even small fluctuations in the labeling mechanism can visibly affect the apparent difficulty of the task. By contrast, corpora such as open_subtitles_corpus and plwiki contain much richer positive support, which makes them better suited for comparing method behaviour across labelling strategies.

4. Stability of Non-SCAR Labelling

To pre-assess stability of non-SCAR labelling, we repeated label generation five times per dataset using different random seeds (11, 22, 33, 44, 55) and fixed target value $c_{calc} = 0.2$. The seed sequence {11, 22, 33, 44, 55} was chosen as a simple, evenly-spaced arithmetic progression to ensure reproducibility and avoid any implicit bias that could arise from

selecting seeds based on prior experimental outcomes. We report two quantities per method: (i) achieved class-prior proxy $c = \mathbb{E}[S \mid Y = 1]$ after labelling, and (ii) absolute deviation from the target, $|c_{calc} - c|$. We additionally report wall-clock labelling time. The experiment covers all six preprocessed SpeakLeash datasets used in the benchmark.

Table 7 summarizes results as mean $\pm$ standard deviation over five runs. We use shortened names in the table: `mvc` and `classic`. The `mvc` variant is a rank-based non-SCAR mechanism calibrated to match a target labeling rate and was used previously in [2]. The `classic` variant uses logistic propensity calibration to match the target class-prior proxy and follows prior PU simulation practice [3]. As expected, `classic` is stochastic and shows non-zero variance across seeds, while `mvc` is deterministic for fixed data and therefore has close to zero variance in the achieved c and deviation. In methodological terms, `classic` reflects a widely used non-SCAR simulation pattern in the PU-learning literature, whereas `mvc` is the authors' original rank-based approach introduced in our earlier work [2].

The two strategies also differ in their practical cost and calibration behaviour. The average runtime of the `classic` generator is substantially higher on large corpora, most clearly on `plwiki`, where the mean labelling time exceeds 30 seconds because the propensity grid search must evaluate a much larger positive pool. MVC is cheaper and fully deterministic, but it can undershoot the target c_{calc} when the two highest-variance features are not sufficient to express the desired selection propensity.

Table 7. Stability of non-SCAR labelling on six preprocessed datasets ($c_{calc} = 0.2$, five seeds).

Dataset	Method	Achieved c	$ c_{calc} - c $	Time [s]
ISAP_corpus	<code>classic</code>	0.188 ± 0.012	0.013 ± 0.011	0.355 ± 0.028
ISAP_corpus	<code>mvc</code>	0.117 ± 0.000	0.083 ± 0.000	0.130 ± 0.004
job_offers_pl_corpus	<code>classic</code>	0.199 ± 0.001	0.001 ± 0.001	8.574 ± 0.198
job_offers_pl_corpus	<code>mvc</code>	0.166 ± 0.000	0.034 ± 0.000	1.294 ± 0.025
open_subtitles_corpus	<code>classic</code>	0.198 ± 0.002	0.002 ± 0.002	0.288 ± 0.015
open_subtitles_corpus	<code>mvc</code>	0.177 ± 0.000	0.023 ± 0.000	0.078 ± 0.005
plwiki	<code>classic</code>	0.200 ± 0.001	0.001 ± 0.000	30.714 ± 10.809
plwiki	<code>mvc</code>	0.088 ± 0.000	0.112 ± 0.000	2.253 ± 0.131
ulotki_medyczne	<code>classic</code>	0.187 ± 0.007	0.013 ± 0.007	0.057 ± 0.011
ulotki_medyczne	<code>mvc</code>	0.074 ± 0.000	0.126 ± 0.000	0.018 ± 0.001
wolne_lektury_corpus	<code>classic</code>	0.200 ± 0.007	0.006 ± 0.003	0.037 ± 0.005
wolne_lektury_corpus	<code>mvc</code>	0.179 ± 0.000	0.021 ± 0.000	0.014 ± 0.003

5. Benchmark Results

In this section, all reported metrics are computed per dataset from a model trained once per seed on the union of training subsets. We evaluated all six PU methods across six SpeakLeash datasets using both non-SCAR labelling strategies: `mvc` and `classic`. Each configuration was run for three target labelling rates, $c_{calc} \in \{0.2, 0.5, 0.8\}$, and ten random seeds. We report AUC, PR-AUC, and F1 as the metrics in the Tables 1–12 of supplementary material³, while the main text summarizes mean F1 aggregated over the three c_{calc} values. We focus on the F1 score because our goal is to ensure clear class separation rather than to optimize decision thresholds, which is typically impractical in LLM training due to its substantial computational cost. As AUC reflects performance across a range of thresholds, we use it only as a supportive, illustrative metric in this analysis.

Tables 8 and 9 provide side-by-side summaries for the two strategies. The split makes strategy-dependent behavior explicit: for example, `lassojoint` achieves notably

³ https://github.com/kapacc/pullm/blob/master/paper/supplement_mdpi.pdf

high F1 under classic on plwiki (0.491) and ulotki_medyczne (0.665). The spy method demonstrates strong results on plwiki under both strategies and on ulotki_medyczne under classic. For plwiki under classic labelling, LassoJoint clearly dominates PR-AUC (0.841/0.718/0.851), remaining well above Spy (0.407 across all c_{calc}) and cluster-based variants, which supports the view that propensity-aware modelling can materially improve positive-class retrieval quality on this corpus (Table 10 in the Supplementary Material).

To check whether the two labelling strategies differ in a consistent way, we compared all matched classic and MVC runs with a paired Wilcoxon test over 1080 observations. The resulting p -value is 0.126, which does not support a global strategy advantage. This is consistent with the dataset-specific pattern in the tables: some corpora favor classic labelling, while others are better served by MVC or by different methods altogether.

Table 8. Benchmark Results Summary (mvc): average F1 over $c_{calc} \in \{0.2, 0.5, 0.8\}$.

Method	ISAP	job_offers	open_subtitles	plwiki	ulotki	wolne_lektury
Naive	0.000	0.000	0.308	0.019	0.000	0.019
Clust	0.006	0.139	0.628	0.270	0.055	0.233
S-LassClust	0.006	0.202	0.632	0.412	0.062	0.278
NS-LassClust	0.006	0.202	0.639	0.412	0.062	0.278
LassoJoint	0.001	0.199	0.580	0.651	0.005	0.223
Spy	0.019	0.192	0.014	0.561	0.103	0.544

Table 9. Benchmark Results Summary (classic): average F1 over $c_{calc} \in \{0.2, 0.5, 0.8\}$.

Method	ISAP	job_offers	open_subtitles	plwiki	ulotki	wolne_lektury
Naive	0.000	0.003	0.329	0.001	0.002	0.110
Clust	0.000	0.075	0.764	0.025	0.017	0.089
S-LassClust	0.000	0.152	0.767	0.060	0.033	0.138
NS-LassClust	0.000	0.125	0.746	0.055	0.027	0.134
LassoJoint	0.000	0.467	0.777	0.491	0.665	0.527
Spy	0.019	0.269	0.018	0.578	0.145	0.556

6. Discussion

The results show that the most reliable conclusions are method- and corpus-specific rather than global. LassoJoint is the strongest general-purpose choice under the classic mechanism, especially for job_offers_pl_corpus, plwiki, and ulotki_medyczne, while Spy is more stable when the data are highly imbalanced or when the feature signal is sparse. Under MVC, the ranking changes more often: Spy remains competitive on ISAP_corpus, ulotki_medyczne, and wolne_lektury_corpus, while cluster-based methods can be preferable for job_offers_pl_corpus.

The failure modes are also informative. On ISAP_corpus, many methods reach F1 values close to zero because the class prior is extremely small (0.84%) and the 22 available stylometric features do not cleanly separate legal high-quality text from the rest of the corpus. This is not a random artifact of one split: the same pattern appears across seeds and both labelling strategies. The near-zero F1 on ISAP_corpus is therefore not a sign that PU methods are intrinsically broken, but rather a consequence of extreme class imbalance combined with a feature space that captures surface-level stylometric properties rather than domain-specific legal quality signals. In other words, the legal domain is a hard case for low-dimensional PU filtering, and the benchmark correctly exposes that limitation. This picture is consistent with recent theoretical analyses of PU learning under extreme label scarcity, such as the ScalePU framework of Dai et al. [19], which highlights how generalization

bounds in PU risk estimation are dominated by the positive-term complexity when very few labeled positives are available. The kernel-embedding approach of Mielniczuk et al. [24] suggests that prior-shift-aware estimation could further improve calibration in such imbalanced regimes, and we leave this extension for future work.

From a practical perspective, the method choice should depend on the data regime. If the corpus is moderately balanced and the goal is a strong high-quality filter, `LassoJoint` is usually the first candidate. If the corpus is very noisy, highly imbalanced, or the selection mechanism is likely weakly informative, `Spy` is a safer baseline. If the selection mechanism itself is being stress-tested or simulated, the difference between classic and MVC should be treated as part of the benchmark design rather than as a single fixed truth.

7. Limitations

This study has several limitations. First, the benchmark uses only the 22 stylometric features currently available in `SpeakLeash`, so it does not test whether richer representations would change the ranking of the methods. Second, we intentionally discard `MEDIUM` documents to keep the task binary and reproducible, which means the setup is an approximation of real filtering rather than a full ternary quality assessment. Third, the current benchmark logs capture wall-clock runtime for model fitting and evaluation together; isolated inference timing is not yet measured separately. Fourth, the reported experiments use one train-test split per seed rather than full cross-validation, so the results should be read as stable benchmarking evidence rather than a formal estimate of out-of-sample performance under all possible splits. Finally, deep generative PU models remain outside the present comparison because they would require a substantially different computational budget and training protocol.

8. Conclusion

This work demonstrates that PU learning can be used as a practical and reproducible framework for quality classification of Polish texts intended for LLM training. We implemented an end-to-end Python pipeline that separates preprocessing, non-SCAR labelling, and benchmark evaluation, and we validated six PU methods across six heterogeneous `SpeakLeash` datasets.

The main empirical conclusion is that there is no single globally best PU method across all domains and labelling strategies. Performance is strongly strategy- and dataset-dependent. Under `classic` labelling, `lassojoint` achieves the strongest overall results on most datasets, while under `mvc` the best choice shifts more often toward `spy` or cluster-based methods depending on the corpus. The paired Wilcoxon test over matched runs did not show a universal difference between the two labelling strategies, so the relevant conclusion is not that one mechanism always wins but that the preferred method depends on the corpus and on how the positive labels are generated. These results can be verified and compared with others in the supplementary material.⁴

The obtained results may be related to the findings of Ociepa et al.[10], for whom one component of the study involved separating text-quality classes. They reported an F1 score of 0.84 (on test subset about 1000 samples) using an XGBoost model; however, their experiments were conducted on fully labeled samples (about 9000 samples). Moreover, they relied on 266 stylometric features, whereas the present work uses only the 22 features directly available in the `SpeakLeash` repository, which substantially simplifies the computational pipeline. Additional advantages of our approach include scalability without manual annotation of negative examples: PU learning, by design, operates without costly

⁴ https://github.com/kapacc/pullm/blob/master/paper/supplement_mdpi.pdf

labeling of the LOW class, which is a key operational benefit when filtering large corpora. Furthermore, methods such as LassoJoint allow modeling of propensity, acknowledging that the probability of a document being labeled depends on its characteristics – a realistic assumption for SpeakLeash annotation systems.

Notably, a substantial portion of the SpeakLeash corpora carries intermediate quality labels (MEDIUM), which are currently discarded in binary setups. As discussed by the SpeakLeash team at conferences, this MEDIUM subset represents untapped potential for LLM training after refined text separation. PU learning directly addresses this scenario: documents labeled MEDIUM may contain genuine positives that remain unlabeled, making the problem structurally equivalent to the single-scenario PU setting where the true positive class is partially observed.

In practical terms, the benchmark suggests the following guidance. Use LassoJoint when the corpus has enough signal for propensity-aware feature selection and when strong retrieval of high-quality texts matters most. Use Spy when the corpus is extremely imbalanced or the feature space is too weak for sparse propensity estimation. Treat cluster-based methods as reasonable middle-ground baselines, especially when the selection mechanism itself may be coarse. Future work should extend the feature set, isolate inference timing, and add deeper PU baselines before drawing broader conclusions about deployment at web scale.

1. SpeakLeash Team. SpeakLeash: Agnostic Dataset Library for the Polish Language. Version 0.3.51; PyPI: Warsaw, Poland, 2024. Available online: <https://pypi.org/project/speakleash/> (accessed on 3 July 2026).
2. Furmańczyk, K.; Paczutkowski, K. A Proposal for PU Classification under Non-SCAR Using Clustering and Logistic Model. In *Proceedings of the 22nd International Conference on Modeling Decisions for Artificial Intelligence (MDAI 2025)*; Torra, V., Narukawa, Y., Domingo-Ferrer, J., Eds.; Springer: Valencia, Spain, 2025. <https://doi.org/10.48550/arXiv.2604.17130>
3. Furmańczyk, K.; Mielniczuk, J.; Rejchel, W.; Teisseyre, P. Double Logistic Regression Approach to Biased Positive-Unlabeled Data. In *ECAI 2023: 26th European Conference on Artificial Intelligence*; Frontiers in Artificial Intelligence and Applications, Vol. 372; IOS Press: Amsterdam, The Netherlands, 2023; pp. 764–771. <https://doi.org/10.3233/FAIA230342>
4. Bekker, J.; Davis, J. Learning from Positive and Unlabeled Data: A Survey. *Mach. Learn.* **2020**, *109*, 719–760. <https://doi.org/10.1007/s10994-020-05877-5>
5. Liu, B.; Lee, W.S.; Yu, P.S.; Li, X. Partially Supervised Classification of Text Documents. In *Proceedings of the 19th International Conference on Machine Learning (ICML 2002)*; Morgan Kaufmann: San Francisco, CA, USA, 2002; pp. 387–394.
6. Fusilier, D.H.; Montes-y-Gómez, M.; Rosso, P.; Cabrera, R.G. Detecting Positive and Negative Deceptive Opinions Using PU-Learning. *Inf. Process. Manag.* **2015**, *51*, 433–443. <https://doi.org/10.1016/j.ipm.2014.11.001>
7. Nigam, K.; McCallum, A.K.; Thrun, S.; Mitchell, T. Text Classification from Labeled and Unlabeled Documents Using EM. *Mach. Learn.* **2000**, *39*, 103–134. <https://doi.org/10.1023/A:1007692713085>
8. Penedo, G.; Kydlíček, H.; Ben Allal, L.; Lozhkov, A.; Mitchell, M.; Raffel, C.; Von Werra, L.; Wolf, T. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024), Track on Datasets and Benchmarks*; Curran Associates: Red Hook, NY, USA, 2024. <https://arxiv.org/abs/2406.17557>
9. Li, R.; Wei, Y.; Zhang, M.; Yu, N.; Hu, H.; Peng, H. ScalingFilter: Assessing Data Quality Through the Lens of Scaling Laws. *arXiv* **2024**, arXiv:2408.08310. <https://arxiv.org/abs/2408.08310>
10. Ociepa, K.; Flis, Ł.; Wróbel, K.; Gwoździej, A.; Kinas, R. Bielik 7B v0.1: A Polish Language Model—Development, Insights, and Evaluation. *Comput. Sci.* **2025**, *26*, 131–161. <https://doi.org/10.7494/csci.2025.26.4.7689>

11. Ociepa, K.; Flis, Ł.; Kinas, R.; Wróbel, K.; Gwoździej, A. Bielik 11B v3: Multilingual Large Language Model for European Languages. *arXiv* **2025**, arXiv:2601.11579. <https://arxiv.org/abs/2601.11579>
12. Wawrzęńczyk, A.; Mielniczuk, J. One-Class Classification Approach to Variational Learning from Biased Positive-Unlabeled Data. In *ECAI 2023: 26th European Conference on Artificial Intelligence*; Frontiers in Artificial Intelligence and Applications, Vol. 372; IOS Press: Amsterdam, The Netherlands, 2023. <https://home.ipipan.waw.pl/j.mielniczuk/VAEPUCC.pdf>
13. Furmańczyk, K.; Paczutkowski, K.; Dudziński, M.; Dziewa-Dawidczyk, D. Classification Methods Based on Fitting Logistic Regression to Positive and Unlabeled Data. In *Computational Science—ICCS 2022*; Lecture Notes in Computer Science, Vol. 13350; Springer: Cham, Switzerland, 2022; pp. 33–46. https://doi.org/10.1007/978-3-031-08751-6_3
14. Hou, M.; Chaib-draa, B.; Li, C.; Zhao, Q. Generative Adversarial Positive-Unlabeled Learning. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI 2018)*; IJCAI Organization: Stockholm, Sweden, 2018; pp. 2255–2261. <https://doi.org/10.24963/ijcai.2018/312>
15. Emami Gohari, A.; Patel, D.; Bhattacharjee, B.; Belgodere, B.; Braz, F.A.; Chaudhary, V.; Dalmia, S.; Fadnis, K.; Ganti, R.; Ghosh, P.; et al. GneissWeb: Preparing High Quality Data for LLMs at Scale. In *Proceedings of the 14th International Conference on Learning Representations (ICLR 2026)*; OpenReview: Vienna, Austria, 2026. <https://openreview.net/forum?id=NRWUAo075J>
16. Wang, J.; Garg, S.; Schneider, A. Accessible, Realistic, and Fair Evaluation of Positive-Unlabeled Learning Algorithms. In *Proceedings of the 14th International Conference on Learning Representations (ICLR 2026)*; OpenReview: Vienna, Austria, 2026. <https://openreview.net/forum?id=5R11h5o44C>
17. Negoifa, V.-A.; Masala, M.; Rebedea, T. Improving Romanian LLM Pretraining Data Using Diversity and Quality Filtering. In *Proceedings of the Second Workshop on Language Models for Low-Resource Languages (LoResLM 2026)*; Association for Computational Linguistics: Rabat, Morocco, 2026; pp. 140–148. <https://doi.org/10.18653/v1/2026.loreslm-1.13>
18. Nait Saada, T.; Béthune, L.; Klein, M.; Grangier, D.; Cuturi, M.; Ablin, P. The Data-Quality Illusion: Rethinking Classifier-Based Quality Filtering for LLM Pretraining. In *Proceedings of the 14th International Conference on Learning Representations (ICLR 2026)*; OpenReview: Vienna, Austria, 2026. Available online: <https://openreview.net/forum?id=vSBACt34gS> (accessed on 3 July 2026).
19. Dai, Y.; Li, X.; Wang, W.; Li, C.; Niu, G.; Sugiyama, M. Positive-Unlabeled Learning with Extreme Scarcity of Labeled Positives. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*; PMLR: 2026. Available online: <https://icml.cc/virtual/2026/poster/61529> (accessed on 3 July 2026).
20. Liu, S.; Chang, Y.; Cheng, M.; Tian, Q.; Li, P. In-Context Positive-Unlabeled Learning. *arXiv* **2026**, arXiv:2605.05591. <https://doi.org/10.48550/arXiv.2605.05591>
21. Mansouri, F.; Zilles, S.; Ben-David, S. Active Learning from Positive and Unlabeled Examples. *arXiv* **2026**, arXiv:2602.02081. <https://doi.org/10.48550/arXiv.2602.02081>
22. Ociepa, K.; Flis, Ł.; Kinas, R.; Wróbel, K.; Gwoździej, A. Advancing Polish Language Modeling through Tokenizer Optimization in the Bielik v3 7B and 11B Series. *arXiv* **2026**, arXiv:2604.10799. <https://doi.org/10.48550/arXiv.2604.10799>
23. Langlais, P.-C.; Chizhov, P.; Arnett, C.; Hinostroza, C.; Nee, M.; Jones, E.; Girard, I.; Mach, D.; Stasenko, A.; Yamshchikov, I. Common Corpus: The Largest Collection of Ethical Data for LLM Pre-Training. In *Proceedings of the 14th International Conference on Learning Representations (ICLR 2026)*; OpenReview: Vienna, Austria, 2026. <https://openreview.net/forum?id=XeAvoKDon0>
24. Mielniczuk, J.; Rejchel, W.; Teisseire, P. Prior Shift Estimation for Positive Unlabeled Data through the Lens of Kernel Embedding. *arXiv* **2026**, arXiv:2502.21194v3. <https://doi.org/10.48550/arXiv.2502.21194>